

ACT01. AUDITORÍA

Contexto

Teniendo en cuenta los conceptos vistos en clase sobre la programación segura a la hora de desarrollar aplicaciones. Realiza la auditoría y la corrección de la aplicación C# de Gestión de usuarios y partidas.

PARTE 1 – Realización de la auditoría

Encuentra y documenta las vulnerabilidades existentes en la aplicación. Recuerda, que vuestro rol para realizar esta parte es de auditor de seguridad, no de desarrollador. A continuación, tenéis la ficha de auditoría a realizar para cada fallo detectado.

Resumen de la auditoría

Se han encontrado **12 vulnerabilidades** en el código original. Las líneas indicadas son las del código original, antes de corregirlo.

7 gravedad alto	3 gravedad medio	2 gravedad bajo
----------------------------------	-----------------------------------	----------------------------------

Nº	Vulnerabilidad	Fichero	Gravedad
1	Inyección SQL en la búsqueda de usuarios	BaseDatosUsuarios.cs	ALTO
2	Panel de administración sin control de acceso	Program.cs	ALTO
3	Contraseñas escritas en el código y mostradas al arrancar	Program.cs	ALTO
4	Clave de la API escrita en el código	RedService.cs	ALTO
5	Contraseñas guardadas con MD5 sin sal	AuthService.cs	ALTO
6	Token de sesión fácil de adivinar	AuthService.cs	ALTO
7	Token de sesión mostrado en pantalla	AuthService.cs, Program.cs	MEDIO
8	Sesión guardada en un fichero sin cifrar	AuthService.cs	MEDIO

Nº	Vulnerabilidad	Fichero	Gravedad
9	Comunicación sin cifrar y clave en la URL	RedService.cs	ALTO
10	No se validan los datos al registrarse	AuthService.cs, Program.cs	MEDIO
11	Errores que muestran detalles internos	Program.cs, RedService.cs	BAJO
12	Consultas SQL mostradas por pantalla	BaseDatosUsuarios.cs	BAJO

Fichas de auditoría

FALLO 1 Inyección SQL en la búsqueda de usuarios **ALTO**

Línea y fichero en el que está ubicado el error:	Datos/BaseDatosUsuarios.cs Líneas 30-34 (método BuscarPorNombre). Línea 32: "... WHERE nombre = '{nombreBuscado}'"
Según los apuntes, ¿qué antipatrón es?	Inyección SQL. Se concatena lo que escribe el usuario dentro de la consulta en lugar de usar una consulta parametrizada.
Si fueras un atacante, ¿cómo lo utilizarías?	En la opción 3 escribiría ' OR '1'='1'. La condición siempre es verdadera y la app devuelve al usuario admin sin saber su nombre. En una base de datos real podría leer o borrar tablas enteras.
Indica el nivel de gravedad: alto, medio, bajo	ALTO Permite robar cuentas, datos o tomar el control.

FALLO 2 Panel de administración sin control de acceso **ALTO**

Línea y fichero en el que está ubicado el error:	Program.cs Líneas 44-46 (case "5") y 146-154 (PanelAdministracion). La línea 84 solo oculta la opción del menú.
Según los apuntes, ¿que antipatrón es?	Control de acceso roto / seguridad por oscuridad. Se esconde la opción, pero el método no comprueba si el usuario tiene permiso.
Si fueras un atacante, ¿cómo lo utilizarías?	Sin iniciar sesión, escribiría 5 en el menú. Entra al panel de administración y ve la lista de todos los usuarios y sus roles.
Indica el nivel de gravedad: alto, medio, bajo	ALTO Permite robar cuentas, datos o tomar el control.

FALLO 3 Contraseñas escritas en el código y mostradas al arrancar **ALTO**

Línea y fichero en el que está ubicado el error:	Program.cs Línea 19: muestra admin/admin1234 en pantalla. Líneas 72-73: contraseñas escritas en el código.
Según los apuntes, ¿que antipatrón es?	Credenciales hardcodeadas y exposición de información sensible. Además son contraseñas débiles.
Si fueras un atacante, ¿cómo lo utilizarías?	Solo con abrir la aplicación ve la contraseña del administrador e inicia sesión como admin. Si el código está en GitHub, cualquiera puede verlas.
Indica el nivel de gravedad: alto, medio, bajo	ALTO Permite robar cuentas, datos o tomar el control.

FALLO 4 Clave de la API escrita en el código

Línea y fichero en el que está ubicado el error:	Servicios/RedService.cs Línea 9: <code>ApiKey = "sk_live_51Hj..."</code>
Según los apuntes, ¿que antipatrón es?	Secretos hardcodeados en el código fuente.
Si fueras un atacante, ¿cómo lo utilizarías?	Leería el código en GitHub o descompilaría el .exe (con ILSpy, por ejemplo) para sacar la clave real (sk_live) y usar la API en nombre de la app: subir puntuaciones falsas, gastar su cuota, etc.
Indica el nivel de gravedad: alto, medio, bajo	ALTO Permite robar cuentas, datos o tomar el control.

FALLO 5 Contraseñas guardadas con MD5 sin sal

Línea y fichero en el que está ubicado el error:	Servicios/AuthService.cs Líneas 56-61 (CalcularHash) usan MD5. Línea 42: compara los hashes con !=.
Según los apuntes, ¿que antipatrón es?	Criptografía débil u obsoleta. MD5 está roto y no usa sal para guardar contraseñas.
Si fueras un atacante, ¿cómo lo utilizarías?	Si consigue los datos de los usuarios, busca los hashes en tablas arcoíris o los rompe por fuerza bruta en segundos. Dos usuarios con la misma contraseña tienen el mismo hash.
Indica el nivel de gravedad: alto, medio, bajo	ALTO Permite robar cuentas, datos o tomar el control.

FALLO 6 Token de sesión fácil de adivinar

Línea y fichero en el que está ubicado el error:	Servicios/AuthService.cs Líneas 63-67 (GenerarTokenSesion): <code>new Random().Next(100000, 999999)</code>
Según los apuntes, ¿que antipatrón es?	Aleatoriedad insegura. Usa <code>Random</code> , que no es criptográfico, y un token muy corto.
Si fueras un atacante, ¿cómo lo utilizarías?	Solo hay 900.000 tokens posibles: con un script los probaría todos hasta acertar uno válido. Con ese token se haría pasar por otro usuario (secuestro de sesión).
Indica el nivel de gravedad: alto, medio, bajo	ALTO Permite robar cuentas, datos o tomar el control.

FALLO 7 Token de sesión mostrado en pantalla

Línea y fichero en el que está ubicado el error:	Servicios/AuthService.cs – línea 49 ([LOG] ... token) Program.cs – línea 143 (VerPerfil)
Según los apuntes, ¿que antipatrón es?	Exposición de información sensible en logs y en la salida por pantalla.
Si fueras un atacante, ¿cómo lo utilizarías?	Mirando la pantalla de otra persona o leyendo los logs, copiaría su token y lo usaría para suplantar su sesión.
Indica el nivel de gravedad: alto, medio, bajo	MEDIO Facilita el robo de sesiones o datos en ciertas condiciones.

FALLO 8 Sesión guardada en un fichero sin cifrar

Línea y fichero en el que está ubicado el error:	Servicios/AuthService.cs Línea 51 (llamada) y líneas 69-72 (GuardarSesionEnDisco). Escribe <code>usuario:token</code> en <code>sesion.txt</code> .
--	---

Según los apuntes, ¿que antipatrón es?	Almacenamiento inseguro de datos sensibles en texto plano.
Si fueras un atacante, ¿cómo lo utilizarías?	En un ordenador compartido abriría <code>sesion.txt</code> y tendría el usuario y el token del último que inició sesión.
Indica el nivel de gravedad: alto, medio, bajo	MEDIO Facilita el robo de sesiones o datos en ciertas condiciones.

FALLO 9 Comunicación sin cifrar y clave en la URL

Línea y fichero en el que está ubicado el error:	Servicios/RedService.cs Línea 10: <code>http://</code> (sin cifrar). Línea 28: <code>api_key</code> y usuario dentro de la URL. Línea 30: imprime la URL completa.
Según los apuntes, ¿que antipatrón es?	Transmisión de datos sin cifrar (HTTP) y datos sensibles en la URL.
Si fueras un atacante, ¿cómo lo utilizarías?	Conectado a la misma wifi (man-in-the-middle) capturaría la petición y vería la <code>api_key</code> . Además, la URL con la clave sale por pantalla y queda guardada en los logs.
Indica el nivel de gravedad: alto, medio, bajo	ALTO Permite robar cuentas, datos o tomar el control.

FALLO 10 No se validan los datos al registrarse

Línea y fichero en el que está ubicado el error:	Servicios/AuthService.cs – líneas 19-31 (Registrar) Program.cs – líneas 93-102
Según los apuntes, ¿que antipatrón es?	Falta de validación de entrada (confiar en lo que escribe el usuario).

Si fueras un atacante, ¿cómo lo utilizarías?	Registraría usuarios vacíos o un segundo "admin" para confundir a los demás. También se aceptan contraseñas como "1", que se adivinan al instante.
Indica el nivel de gravedad: alto, medio, bajo	MEDIO Facilita el robo de sesiones o datos en ciertas condiciones.

FALLO 11 Errores que muestran detalles internos

Línea y fichero en el que está ubicado el error:	Program.cs – líneas 58-62 (ex.ToString()) Servicios/RedService.cs – línea 21 (ex.Message)
Según los apuntes, ¿qué antipatrón es?	Manejo inadecuado de errores (fuga de información).
Si fueras un atacante, ¿cómo lo utilizarías?	Provocaría errores a propósito para ver la traza completa: nombres de clases, métodos, rutas y líneas. Así entiende cómo está hecha la app y prepara otros ataques.
Indica el nivel de gravedad: alto, medio, bajo	BAJO Da pistas al atacante, pero no basta por sí solo.

FALLO 12 Consultas SQL mostradas por pantalla

Línea y fichero en el que está ubicado el error:	Datos/BaseDatosUsuarios.cs Línea 41: <code>Console.WriteLine(\$"[DB] {consulta}")</code>
Según los apuntes, ¿qué antipatrón es?	Fuga de información (mensajes de depuración que se dejan en el programa final).
Si fueras un atacante, ¿cómo lo utilizarías?	Vería la consulta exacta que se ejecuta y sabría cómo escribir la inyección SQL perfecta (fallo 1).
Indica el nivel de gravedad: alto, medio, bajo	BAJO Da pistas al atacante, pero no basta por sí solo.

PARTE 2 – Corrección de los errores

Siguiendo los apuntes, corrige los errores detectados en el código. Modifica el código de la aplicación entregada.

Todas las modificaciones están comentadas en el código con `// ARREGLO:` (qué se ha cambiado) y `// Motivo:` (por qué).

Nº	Fichero	Corrección aplicada
1	BaseDatosUsuarios.cs	BuscarPorNombre compara el nombre directamente como dato (como una consulta parametrizada). Eliminado el método que interpretaba la consulta.
2	Program.cs	PanelAdministracion comprueba que hay sesión y rol admin antes de mostrar nada.
3	Program.cs	Contraseñas leídas de las variables de entorno ADMIN_PASSWORD y ANA_PASSWORD. Eliminada la línea que las mostraba.
4	RedService.cs	La clave se lee de la variable de entorno API_KEY.
5	AuthService.cs	PBKDF2 con SHA-256, sal aleatoria y 100.000 iteraciones. Los hashes se comparan con FixedTimeEquals.
6	AuthService.cs	Token generado con RandomNumberGenerator (32 bytes aleatorios).
7	AuthService.cs, Program.cs	Eliminados el log del token y la línea del perfil que lo mostraba.
8	AuthService.cs	Eliminado el método GuardarSesionEnDisco.
9	RedService.cs	HTTPS, clave en la cabecera X-API-Key y usuario codificado con Uri.EscapeDataString. La URL ya no se imprime.
10	AuthService.cs	Nombre no vacío ni repetido y contraseña de al menos 8 caracteres.
11	Program.cs, RedService.cs	Mensaje de error genérico. Solo se muestran los mensajes de validación propios.
12	BaseDatosUsuarios.cs	Eliminado el Console.WriteLine de las consultas.

ENTREGA:

- El código compilado y corregido (recuerda comentar las modificaciones que hayas realizado).

<https://github.com/mileagaabriel-dam1/CIBERSEGURIDAD>